

MICRO PROJECT REPORT

On

STUDENT RECORD MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology in Computer Science & Engineering

Submitted by:

Vivek Narwariya (BTCS26O1151)

Jatin Mittal (BTCS26O1056)

Under the Guidance of:

Prof. Jaimala Jha

Department of Computer Science & Engineering

Madhav Institute of Technology and Science (MITS), Gwalior

(A Govt. Aided UGC Autonomous & NAAC Accredited Institute)

Session 2026-2027

CERTIFICATE

This is to certify that the micro project entitled "**Student Record Management System**" has been successfully carried out and completed by **Vivek Narwariya (Enrollment No: BTCS26O1151)** and **Jatin Mittal (Enrollment No: BTCS26O1056)** under my supervision and guidance.

This report is being submitted to the Department of Computer Science & Engineering, Madhav Institute of Technology and Science, Gwalior, in partial fulfillment of the requirements for the degree of Bachelor of Technology (B.Tech) during the academic session 2026-2027.

Date: _____

Prof. Jaimala Jha
Project Guide
Dept. of CSE, MITS Gwalior

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to our project guide, **Prof. Jaimala Jha**, for her continuous support, valuable feedback, and proper guidance throughout the development of this micro project. Her knowledge and direction helped us in completing this work successfully.

We also want to extend our sincere thanks to the Head of the Department (Computer Science & Engineering) and the Director of Madhav Institute of Technology and Science (MITS), Gwalior, for providing the necessary computer labs and a great learning environment.

Finally, we are very thankful to our families, friends, and classmates for their motivation and help whenever we faced any difficulties in coding or logic building during the project.

Vivek Narwariya (BTCS26O1151)
Jatin Mittal (BTCS26O1056)

INDEX

1. Introduction	1
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Project Objectives	1
2. System Analysis & Requirements	2
2.1 Technical Concepts Used	2
2.2 Software & Hardware Requirements	2
3. System Design	3
3.1 Data Structures Definition	3
3.2 Logic and Algorithm	3
4. Implementation	4
4.1 Main Menu Operations	4
4.2 Source Code Snippets	4
5. Output and Testing	5
6. Conclusion & Future Scope	6

1. INTRODUCTION

1.1 Project Overview

The Student Record Management System is a simple console-based application created using the C programming language. The main purpose of this software is to store student data easily and retrieve it whenever needed. Instead of writing student details in paper registers, this program uses computer memory and file storage to keep the records safe. It allows users to add new student information, view a list of all students, search for a specific student, and delete records if they are no longer needed.

1.2 Problem Statement

In many educational institutes and small coaching centers, student records are maintained manually in large paper files. This manual system causes several problems:

- **Time Consuming:** Finding one specific student's record out of hundreds of pages takes a lot of time.
- **Data Loss:** Paper registers can easily get damaged, lost, or misplaced over time.
- **Difficult to Update:** Modifying details or deleting a student who left the college requires messy cutting or rewriting in the register.

1.3 Project Objectives

To solve the above problems, we have developed this project with the following objectives in mind:

- To build a digital record system that is fast, reliable, and easy to use.
- To learn and implement file handling concepts in the C programming language for permanent data storage.
- To provide an interactive main menu so that any user with basic computer knowledge can operate it.
- To eliminate the need for paper records, saving stationary costs and physical storage space.

2. SYSTEM ANALYSIS & REQUIREMENTS

2.1 Technical Concepts Used

To build this project, we used several important concepts from the C programming language. Applying these concepts practically was a major part of our learning process.

- **Structures (struct):** We used a struct to group different types of data together. A student has a name (string), roll number (integer), and CGPA (float). A struct allows us to handle all this under one single unit.
- **File Handling:** Normally, C variables lose their data when the program closes. We used standard C file operations like `fopen()`, `fwrite()`, `fread()`, and `fclose()` to store the student structures directly into a file on the hard drive (e.g., `students.dat`).
- **String Manipulation:** We used functions from `<string.h>` such as `strcmp()` to search for specific text strings when looking up student names or courses.
- **Control Statements:** We used `switch-case` statements to build the main menu, and `while` loops to keep the program running until the user chooses to exit.

2.2 Software & Hardware Requirements

Because this is a simple console application, the system requirements are very low. It can run on almost any computer.

Software Requirements:

- **Operating System:** Windows 10/11, Linux, or macOS.
- **Programming Language:** C
- **Compiler:** GCC (GNU Compiler Collection) or MinGW.
- **IDE/Editor:** Dev C++, Code::Blocks, or Visual Studio Code.

Hardware Requirements:

- **Processor:** Basic Intel Core i3 or equivalent (even older processors will work perfectly).
- **RAM:** 2 GB Minimum.
- **Storage:** Less than 50 MB of hard disk space.

3. SYSTEM DESIGN

3.1 Data Structures Definition

The core of the database is the student structure. This dictates exactly what information we are storing for each user. In our C program, the structure is defined as follows:

```
struct Student { int roll_no; char name[50]; char branch[50]; float  
cgpa; };
```

Every time a new student is added, a new instance of this structure is created, populated with user input, and then written as a block of bytes to the data file.

3.2 Logic and Algorithm for Deletion

Adding and viewing records is straightforward. However, deleting a record from a binary file in C requires a specific algorithm. Since we cannot easily remove bytes from the middle of a file, we used the temporary file method.

Step-by-step Algorithm for Delete Operation:

1. Start the delete function.
2. Ask the user to enter the Roll Number of the student they want to delete.
3. Open the main data file `students.dat` in read mode ("rb").
4. Create and open a new temporary file `temp.dat` in write mode ("wb").
5. Use a loop to read records one by one from `students.dat`.
6. Inside the loop, compare the entered Roll Number with the Roll Number of the current record.
7. If the Roll Numbers match, skip writing this record (this effectively removes it).
8. If the Roll Numbers do not match, write the record to `temp.dat`.
9. Close both files after the loop finishes.
10. Delete the original `students.dat` file using the `remove()` function.
11. Rename `temp.dat` to `students.dat` using the `rename()` function.
12. Print "Record Deleted Successfully" and Stop.

4. IMPLEMENTATION

4.1 Main Modules

The program is divided into modular functions to keep the code clean and easy to read. The main modules are:

- `addRecord()`: Takes input from the keyboard and appends it to the file.
- `viewRecords()`: Prints a formatted table layout of all students.
- `searchRecord()`: Finds a student by roll number and displays their details.

4.2 Source Code Snippet

Below is the main implementation code showing the Add Record function and file writing logic.

```
void addRecord() { FILE *fp; struct Student s; fp =
fopen("students.dat", "ab"); if(fp == NULL) { printf("Error opening
file\n"); return; } printf("\n=== Add New Student ===\n"); printf("Enter
Roll Number: "); scanf("%d", &s.roll_no); fflush(stdin); // Clear input
buffer printf("Enter Name: "); gets(s.name); printf("Enter Branch: ");
gets(s.branch); printf("Enter CGPA: "); scanf("%f", &s.cgpa); fwrite(&s,
sizeof(struct Student), 1, fp); printf("Record Added Successfully!\n");
fclose(fp); }
```

In this code, we open the file in "`ab`" (append binary) mode. This ensures that new records are added to the end of the file without deleting old records. The `fwrite()` function is used to write the entire structure block in one go.

5. OUTPUT AND TESTING

System Testing

After writing the code, we compiled it using the GCC compiler and tested it thoroughly to make sure there are no runtime errors. We ran the following tests:

- **Insertion Test:** Added 5 dummy student records to check if the file creates properly. It successfully created `students.dat`.
- **Display Test:** Called the View function. The output displayed neatly in columns.
- **Search Failure Test:** Searched for Roll Number "999" (which did not exist). The system correctly printed "Record Not Found" instead of crashing.

Expected Console Interface

Because this is a command-line application, there are no graphics. The interface looks like this when the program is run:

```
===== STUDENT RECORD MANAGEMENT
===== 1. Add New Student Record 2. View All
Records 3. Search Student 4. Delete Student 5. Exit
===== Enter your choice: 2 ROLL NO NAME
BRANCH CGPA ----- 1056
Jatin Mittal CSE 8.5 1151 Vivek Narwariya CSE 8.8
```

The program uses simple print formatting %-10d %-20s to align the columns properly on the black console screen.

6. CONCLUSION & FUTURE SCOPE

6.1 Conclusion

The "Student Record Management System" micro project was successfully completed and tested. Through this project, we practically applied core C programming concepts, specifically structures, pointers, and file handling. The resulting software successfully replaces the slow, error-prone manual paper system with a fast, digital file-based system. It provides an easy-to-use menu that allows users to perform basic CRUD (Create, Read, Update, Delete) operations effortlessly from the terminal.

6.2 Limitations

While the system works well, it has some limitations:

- It is a local console application, meaning it cannot be accessed over the internet or by multiple teachers at the same time.
- The user interface is text-based and lacks modern graphical elements (GUI).
- There is no password protection system to stop unauthorized people from opening the program and deleting records.

6.3 Future Scope

If we continue to develop this project in the future, we could add the following features:

- Adding an admin login screen with a password using string masking.
- Exporting the student records to a CSV file format so they can be viewed and printed easily using Microsoft Excel.
- Sorting the displayed records automatically based on Roll Number or highest CGPA.

References

- Kanetkar, Y. (2016). *Let Us C*. BPB Publications.
- Class lectures and notes provided by Prof. Jaimala Jha.
- C Programming documentation (Standard C Library).